

WalletTrackers Project Documentation

WalletTrackers Project Documentation

Document version: 1.0

Date: 2026-05-02

Project: WalletTrackers

Platform: Android

1. Overview

WalletTrackers is a personal finance tracking Android application. It helps a user manage accounts, records, budgets, savings goals, debts, monthly bills, credit card statements, SMS-imported transactions, statistics, transfers, and currency conversion.

The application is built with Kotlin and Jetpack Compose. Firebase Authentication and Firestore provide user identity and cloud data storage. The app can read bank SMS messages, classify transactions, and save financial records automatically.

2. Project Scope

The project includes one Android application module:

- * Root project: WalletTrackers
- * Android module: app
- * Application ID: com.example.wallettrackers
- * Minimum SDK: 29
- * Target SDK: 36
- * Compile SDK: 36

The application supports:

- * Authentication by email, Google, and Facebook.
- * First-run onboarding for account discovery and setup.
- * Manual and automatic transaction tracking.
- * Account balance updates.
- * Credit card statement and payment handling.
- * Budget alerts.
- * Monthly bill and subscription detection.
- * Statistics and calendar views.
- * CSV export of records.
- * Biometric app lock.

3. Technology Stack

- * Language: Kotlin
- * UI: Jetpack Compose, Material 3
- * Architecture: MVVM with repository pattern
- * Navigation: Navigation Compose
- * Backend: Firebase Authentication and Firebase Firestore
- * AI: Google Gemini via google-generativeai
- * Networking: Ktor client
- * Charts: Vico
- * Background work: WorkManager
- * Security: Android BiometricPrompt
- * SMS integration: BroadcastReceiver and device SMS reader

WalletTrackers Project Documentation

4. High-Level Architecture

The app is organized into UI screens, ViewModels, repositories, domain models, utility classes, workers, receivers, and service classes.

Primary layers:

- * UI layer: Compose screens in screens/
- * State and business logic: ViewModels in viewmodel/
- * Data access: FirebaseRepository
- * Models: data classes in model/
- * SMS and AI automation: SmsReceiver, SmsViewModel, AiService
- * Notifications and reminders: NotificationHelper, ReminderManager, BillReminderManager, WorkManager workers

5. Application Navigation

The main navigation graph is defined in MainActivity.kt.

Main routes:

- * login: sign in entry point.
- * signup: email registration.
- * onboarding/{userId}: first-run onboarding.
- * home: dashboard and side navigation.
- * add_record?category={category}: manual record creation.
- * all_records: transaction list, edit, delete, category rules, and export.
- * currency_converter: exchange rate conversion.
- * categories: category list.
- * subcategories/{categoryName}: subcategory picker.
- * statistics: charts, insights, and credit statement actions.
- * sms: SMS import/review screen.
- * budget: budget management.
- * transfer: internal account transfer.
- * goals: savings goals.
- * debts: debt tracking.
- * bills: monthly bill and recurring payment detection.
- * calendar: date-based record view.

6. Core Data Model

Account

Represents a wallet, bank account, cash account, or credit card.

Important fields:

- * id
- * name
- * accountType
- * last4Digits
- * amount
- * currency
- * creditLimit
- * billingDay

WalletTrackers Project Documentation

- * isArchived
- * userId

Record

Represents an income or expense transaction.

Important fields:

- * id
- * accountId
- * accountName
- * category
- * amount
- * currency
- * type
- * timestamp
- * balanceAfter
- * smsId
- * comment
- * userId

CreditStatement

Represents a credit card bill statement extracted from SMS or managed in the app.

Budget

Represents a monthly spending limit for a category.

SavingsGoal

Represents a savings target and progress.

Debt

Represents money owed by or to the user.

Bill

Represents a monthly bill or subscription.

Important fields:

- * id
- * name
- * amount
- * currency
- * dayOfMonth
- * category
- * isActive
- * userId

CategoryRule

Represents a merchant keyword to category mapping.

7. Firebase Data Structure

WalletTrackers Project Documentation

The repository stores user-owned documents under:

users/{userId}

Subcollections:

- * accounts
- * records
- * creditStatements
- * budgets
- * savingsGoals
- * debts
- * bills
- * categoryRules

The repository uses Firestore snapshot listeners to keep UI state synchronized in real time.

8. Authentication

The app supports:

- * Google sign-in.
- * Facebook sign-in.
- * Email sign-in.
- * Email sign-up.

After login, first-time users are routed to onboarding. Returning users go to the home screen. Sign-out and account deletion are exposed from the home experience.

9. Onboarding

The onboarding flow prepares a first-time user's app state. It uses device SMS access and account discovery to help initialize tracked accounts.

10. Transaction Management

Users can manually add, edit, and delete records. Adding a normal expense or income updates the linked account balance. Editing and deleting records adjusts account balances where possible.

Special cases:

- * Transfer records use accountName formatted as From Account -> To Account.
- * Credit card payments update both debit and credit accounts when both sides are known.
- * Transactions imported from SMS keep their SMS timestamp.

11. SMS Processing

SmsReceiver listens for incoming SMS messages when permissions are granted. Processing includes:

- * Duplicate detection using smsId.
- * Promotional and declined transaction filtering.
- * Bank SMS detection.
- * Amount, currency, account digits, date, balance, category, and comment extraction.
- * Gemini-based fallback classification.
- * Account matching by last digits.

WalletTrackers Project Documentation

- * Record, statement, ATM withdrawal, card payment, and credit-card-received handling.
- * User notifications after automatic tracking.

SmsViewModel and DeviceSmsReader support reading device SMS for manual import/review workflows.

12. AI Classification

AiService uses Gemini to:

- * Infer a category from SMS text.
- * Analyze unstructured SMS into a structured transaction object.

The AI output is parsed into ExtractedTransaction. The app falls back to keyword-based parsing when AI classification fails.

13. Credit Card Statement and Payment Handling

The app detects statement SMS messages and stores unpaid credit statements. It schedules statement reminders and allows the user to pay statements from a selected debit account.

Credit card payments can arrive as two SMS messages:

- * Debit-side SMS: money leaves a debit account.
- * Credit-side SMS: money arrives to a credit card account.

The receiver links both sides using amount matching and pending-payment storage, then creates or updates a single transfer-style payment record.

14. Monthly Bills and Recurring Detection

The Monthly Bills screen shows detected subscriptions and recurring payments. The current detection logic:

- * Detects paid subscriptions from the last month.
- * Detects recurring non-subscription payments with the same amount, day, and currency across the last two months.
- * Excludes transfers, credit payments, credit records, and Instapay income/outcome.
- * Allows the user to confirm a suggestion as a monthly bill or dismiss it.
- * Schedules bill reminders when a bill is added.

Subscription identification uses the Subscriptions category and known service names such as Netflix, YouTube, Amazon, Spotify, Disney, and Yango.

15. Budgets

Users can create, update, and delete budgets. When a new expense causes category spending to reach 80 percent or more of the budget, the app sends a budget alert notification.

16. Statistics

The statistics screen uses records, accounts, and credit statements to show spending analytics and actionable statement payment controls. The app calculates monthly insight data such as total expense and top spending category.

17. Savings Goals and Debts

WalletTrackers Project Documentation

Savings goals and debts are stored in Firestore and managed by the home ViewModel. Users can create, update, and delete these objects.

18. Transfers

The transfer screen allows moving money between accounts. The app updates both account balances and creates a transfer-style record.

19. Currency Converter

The currency converter uses the Ktor client and exchange-rate API integration to convert between currencies.

20. Notifications and Reminders

The app uses Android notifications and WorkManager-backed reminders for:

- * Transaction alerts.
- * Credit card statement reminders.
- * Monthly bill reminders.
- * Budget alerts.

Notification permission is requested on Android versions that require it.

21. Security and Privacy

Security-related features:

- * Firebase Authentication identifies users.
- * Firestore data is stored under each authenticated user.
- * SMS permissions are requested at runtime.
- * Biometric lock can be enabled by the user.
- * API keys are loaded from local.properties into BuildConfig.

Sensitive areas:

- * SMS data contains personal financial information.
- * Firebase rules must restrict each user to their own data.
- * API keys and google-services.json must not be exposed publicly.

22. Build and Configuration

Build system:

- * Gradle Kotlin DSL
- * Android Gradle Plugin via version catalog
- * Kotlin Compose plugin
- * Google services plugin

Build commands:

```
.\gradlew.bat assembleDebug
.\gradlew.bat test
.\gradlew.bat connectedAndroidTest
```

Required local configuration:

- * app/google-services.json

WalletTrackers Project Documentation

* local.properties containing gemini.api.key=...

23. Known Risks and Recommendations

- * SMS parsing depends on bank message formats and can require ongoing updates.
- * AI classification requires network access and a valid Gemini key.
- * Firestore security rules are not visible in this repository and should be audited.
- * Release signing configuration is not defined in the current Gradle file.
- * Unit test coverage is minimal and should be expanded around SMS parsing, balance updates, recurring detection, and credit payment linking.